

Algorithms for Data Science

Cheatsheet for Exercises 3

Master Theorem. Let $T(n) = aT(n/b) + f(n)$ with constants $a > 0$, $b > 1$, and $f(n)$ being a nondecreasing function. Let $c = \log_b a$.

- **(Case A)** If $f(n) = O(n^{c'})$ for some $c' < c$, then $T(n) = \Theta(n^c)$.
- **(Case B)** If $f(n) = \Theta(n^c)$, then $T(n) = \Theta(n^c \log n)$.
- **(Case C)** If $f(n) = \Omega(n^{c'})$ for some $c' > c$, then^(*) $T(n) = \Theta(f(n))$.

^(*)Note: for Case C, $f(n)$ must also satisfy the regularity condition: there is a constant $k < 1$ such that for all sufficiently large n , we have $af(n/b) \leq kf(n)$. However, in our course, you don't have to learn it.

It is very easy to decide which case holds if $f(n)$ is a polynomial:

Master Theorem “Lite”

Compute $c = \log_b a$ as above. If $f(n) = \Theta(n^d)$ for some constant number d such that

- $\dots d < c$, then Case A holds.
- $\dots d = c$, then Case B holds.
- $\dots d > c$, then Case C holds.

Note: the regularity condition holds automatically for Case C if $f(n) = \Theta(n^d)$.

Nice to Know. If $T(n) = aT(\lfloor n/b \rfloor) + f(n)$ or $T(n) = aT(\lceil n/b \rceil) + f(n)$, then we can simplify and write $T(n) = aT(n/b) + f(n)$. This simplification does not change the solution to $T(n)$.

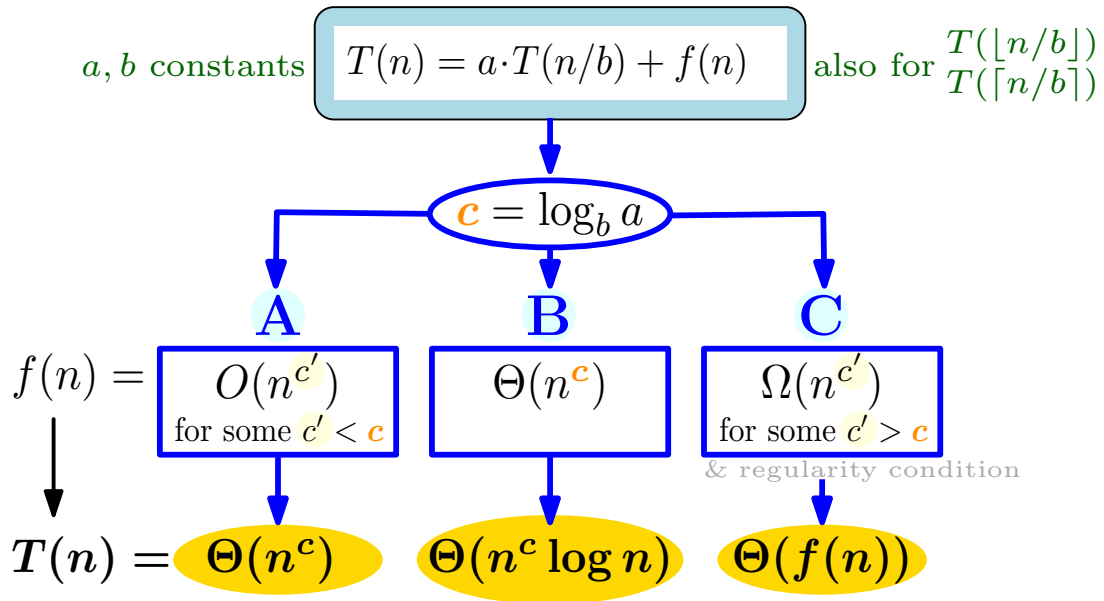
Notation. Recall that

- $\lfloor \cdot \rfloor$ is the *floor* function that rounds a given number *down* to the next integer.
Examples: $\lfloor 1.5 \rfloor = 1$ and $\lfloor 4 \rfloor = 4$.
- $\lceil \cdot \rceil$ is the *ceiling* function that rounds a given number *up* to the next integer.
Examples: $\lceil 1.5 \rceil = 2$ and $\lceil 4 \rceil = 4$.

Summation Rules.

1. (First n integers) $1 + \dots + n = \sum_{i=1}^n i = n(n+1)/2 = \Theta(n^2)$
2. (First n power of twos) $1 + 2 + 2^2 + \dots + 2^n = \sum_{i=0}^n 2^i = 2^{n+1} - 1 = \Theta(2^n)$

Master Theorem



Master Theorem Lite

